

在给予适当署名的前提下，Google 特此允许复制本文中的表格和图片，但仅限用于新闻报道或学术作品。

注意力机制就是你所需要的一切

Attention Is All You Need

Ashish Vaswani* Noam Shazeer* Niki Parmar*
Jakob Uszkoreit* Llion Jones* Aidan N. Gomez*[†]
Łukasz Kaiser* Illia Polosukhin*[‡]

Ashish Vaswani, Google Brain	avaswani@google.com
Noam Shazeer, Google Brain	noam@google.com
Niki Parmar, Google Research	nikip@google.com
Jakob Uszkoreit, Google Research	usz@google.com
Llion Jones, Google Research	llion@google.com
Aidan N. Gomez, 多伦多大学	aidan@cs.toronto.edu
Łukasz Kaiser, Google Brain	lukaszkaizer@google.com
Illia Polosukhin	illia.polosukhin@gmail.com

摘要

当前主流的序列转换模型基于包含编码器和解码器的复杂循环神经网络或卷积神经网络。性能最好的模型还通过注意力机制连接编码器与解码器。我们提出一种新的简单网络架构 **Transformer**，它完全基于注意力机制，彻底去除了循环和卷积。在两项机器翻译任务上的实验表明，这些模型不仅质量更高，还具有更强的可并行性，并且所需训练时间显著更少。我们的模型在 WMT 2014 英语到德语翻译任务上达到 28.4 BLEU，比包括集成模型在内的既有最佳结果高出 2 BLEU 以上。在 WMT 2014 英语到法语翻译任务上，模型在 8 块 GPU 上训练 3.5 天后，以 41.8 BLEU 创下新的单模型最佳成绩，而训练开销仅为文献中最佳模型的一小部分。我们还在大量训练数据和有限训练数据两种条件下，成功地将 **Transformer** 应用于英语成分句法分析，展示了它对其他任务良好的泛化能力。

* 同等贡献。作者顺序随机排列。Jakob 提出用自注意力替代 RNN，并发起了对这一想法的评估。Ashish 与 Illia 设计并实现了最初的 **Transformer** 模型，并对本工作的各个方面都作出了关键贡献。Noam 提出了缩放点积注意力、多头注意力和无参数的位置表示，并成为另一位几乎参与所有细节的成员。Niki 在最初的代码库和 **tensor2tensor** 中设计、实现、调优并评估了无数模型变体。Llion 也试验了新颖的模型变体，负责最初的代码库、高效推理及可视化。Łukasz 和 Aidan 花费了无数个漫长的工作日，设计并实现 **tensor2tensor** 的各个部分，替换了早期代码库，显著改进了结果，并极大加速了我们的研究。

[†] 工作在 Google Brain 任职期间完成。 [‡] 工作在 Google Research 任职期间完成。

第 31 届神经信息处理系统大会 (NIPS 2017)，美国加利福尼亚州长滩。

1 引言

循环神经网络，尤其是长短期记忆网络 [13] 和门控循环神经网络 [7]，已经确立了其作为序列建模与序列转换问题中先进方法的地位，例如语言建模与机器翻译 [35, 2, 5]。此后，大量研究持续推动循环语言模型与编码器—解码器架构的性能边界 [38, 24, 15]。

循环模型通常沿输入和输出序列的符号位置分解计算。它们将这些位置与计算时间步对齐，根据前一个隐藏状态 h_{t-1} 和位置 t 的输入生成隐藏状态序列 h_t 。这种固有的顺序性使得单个训练样本内部无法并行化；随着序列变长，内存限制又会约束跨样本的批处理，因此这一问题尤为突出。近期工作通过分解技巧 [21] 与条件计算 [32]，显著提升了计算效率，其中后者还改善了模型性能。然而，顺序计算这一根本约束仍然存在。

在多种任务中，注意力机制已成为高性能序列建模与序列转换模型不可或缺的组成部分，它能够建模依赖关系，而不受这些依赖在输入或输出序列中距离的影响 [2, 19]。但除少数情况外 [27]，此类注意力机制都与循环网络结合使用。

本文提出 Transformer：一种摒弃循环、完全依赖注意力机制来建立输入与输出之间全局依赖的模型架构。Transformer 支持程度显著更高的并行化，仅在 8 块 P100 GPU 上训练 12 小时，就能在翻译质量上达到新的最佳水平。

2 背景

减少顺序计算也是 Extended Neural GPU [16]、ByteNet [18] 和 ConvS2S [9] 的基础目标。这些模型都以卷积神经网络为基本构件，并行计算所有输入和输出位置的隐藏表示。在这些模型中，建立任意两个输入或输出位置的信号联系所需的操作数，会随位置间距离增长：ConvS2S 为线性增长，ByteNet 为对数增长。这使得远距离位置之间的依赖更难学习 [12]。在 Transformer 中，这一操作数降低为常数，代价则是对注意力加权位置取平均会降低有效分辨率；我们通过第 3.2 节介绍的多头注意力来抵消这一影响。

自注意力有时也称为内部注意力，它通过关联同一序列中的不同位置，计算该序列的表示。自注意力已成功应用于多种任务，包括阅读理解、生成式摘要、文本蕴含和任务无关句子表示的学习 [4, 27, 28, 22]。

端到端记忆网络基于循环注意力机制，而不是与序列对齐的循环机制；它已在简单语言问答和语言建模任务中取得良好表现 [34]。

然而，据我们所知，Transformer 是第一个完全依赖自注意力计算输入和输出表示、而不使用序列对齐 RNN 或卷积的序列转换模型。接下来，我们将介绍 Transformer，阐述使用自注意力的动机，并讨论它相对于文献 [17, 18] 与 [9] 等模型的优势。

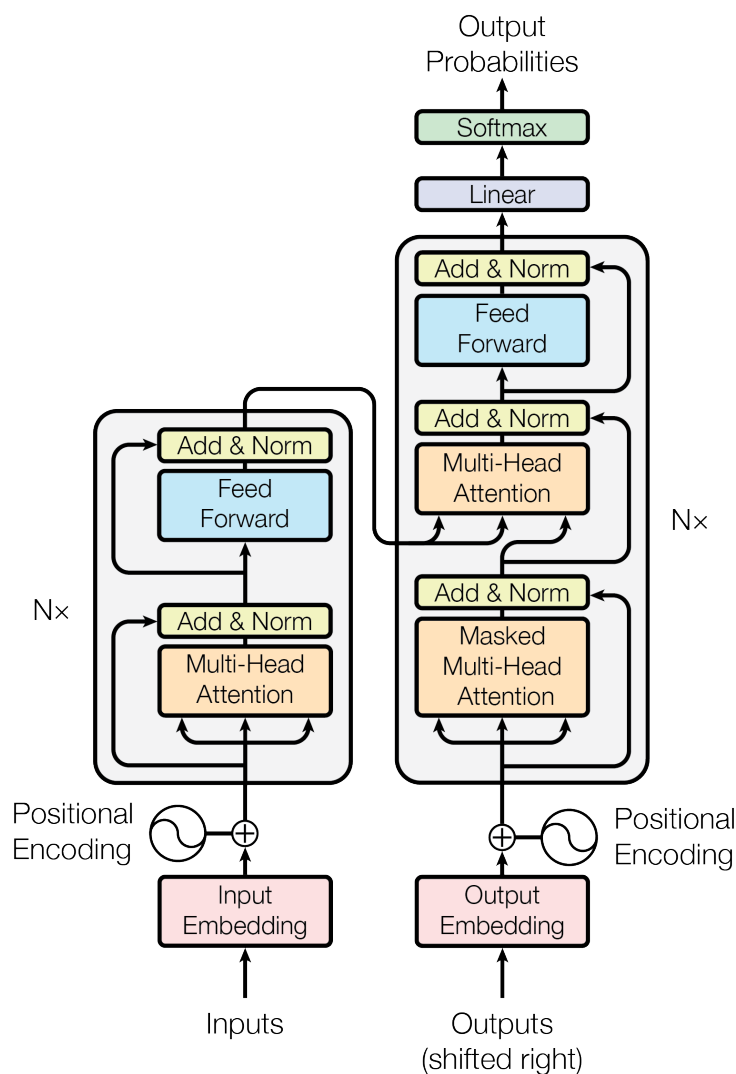


图 1. Transformer 模型架构。

3 模型架构

大多数有竞争力的神经序列转换模型都采用编码器—解码器结构 [5, 2, 35]。在这种结构中，编码器将符号表示的输入序列 $(x_1, \dots, x_n)$ 映射为连续表示序列 $\mathbf{z} = (z_1, \dots, z_n)$ 。给定 $\mathbf{z}$ 后，解码器逐个元素生成符号输出序列 $(y_1, \dots, y_m)$ 。在每一步，模型都是自回归的 [10]：生成下一个符号时，将先前生成的符号作为额外输入。

Transformer 遵循这一总体架构，在编码器和解码器中均使用堆叠的自注意力层以及逐位置全连接层，分别如图 1 的左半部分和右半部分所示。

3.1 编码器与解码器堆栈

编码器：编码器由 $N = 6$ 个相同的层堆叠而成。每层包含两个子层：第一个是多头自注意力机制，第二个是简单的逐位置全连接前馈网络。我们在两个子层的外围分别使用残差连接 [11]，随后执行层归一化 [1]。也就是说，每个子层的输出为 $\text{LayerNorm}(x + \text{Sublayer}(x))$ ，其中 $\text{Sublayer}(x)$ 是子层本身实现的函数。为便于使用这些残差连接，模型中的所有子层以及嵌入层都产生维度为 $d_{\text{model}} = 512$ 的输出。

解码器：解码器也由 $N = 6$ 个相同的层堆叠而成。除每个编码器层中的两个子层之外，解码器还插入第三个子层，对编码器堆栈的输出执行多头注意力。与编码器类似，我们在每个子层外围使用残差连接，随后执行层归一化。我们还修改了解码器堆栈中的自注意力子层，防止某个位置关注其后续位置。这种掩码与输出嵌入向后偏移一个位置的做法结合，确保位置 i 的预测只能依赖于小于 i 的位置上已知的输出。

3.2 注意力

注意力函数可以描述为：将一个查询和一组键—值对映射为一个输出，其中查询、键、值和输出都是向量。输出由各个值的加权和计算而来，分配给每个值的权重，则由查询与对应键之间的相容性函数计算。

3.2.1 缩放点积注意力

我们将所使用的注意力称为“缩放点积注意力”（图 2）。输入由维度为 d_k 的查询和键，以及维度为 d_v 的值组成。我们计算查询与所有键的点积，将每个点积除以 $\sqrt{d_k}$ ，再应用 **softmax** 函数得到各个值的权重。

在实践中，我们同时对一组查询计算注意力函数，将这些查询合并为矩阵 Q 。键和值也分别合并为矩阵 K 和 V 。输出矩阵的计算方式为

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

最常用的两种注意力函数是加性注意力 [2] 和点积（乘性）注意力。除缩放因子 $\frac{1}{\sqrt{d_k}}$ 外，点积注意力与我们的算法完全相同。加性注意力使用一个具有单隐藏层的前馈网络计算相容性函数。虽然二者理论复杂度相近，但在实际应用中，点积注意力速度更快、空间效率更高，因为它可以利用高度优化的矩阵乘法代码实现。

当 d_k 较小时，两种机制表现相近；但当 d_k 较大时，加性注意力优于未经缩放的点积注意力 [3]。我们推测，较大的 d_k 会使点积的数值增大，将 **softmax** 函数推入梯度极小的区域⁴。为抵消这

⁴为说明点积为何会变大，假设 q 和 k 的各分量是均值为 0、方差为 1 的独立随机变量。那么，它们的点积 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的均值为 0，方差为 d_k 。

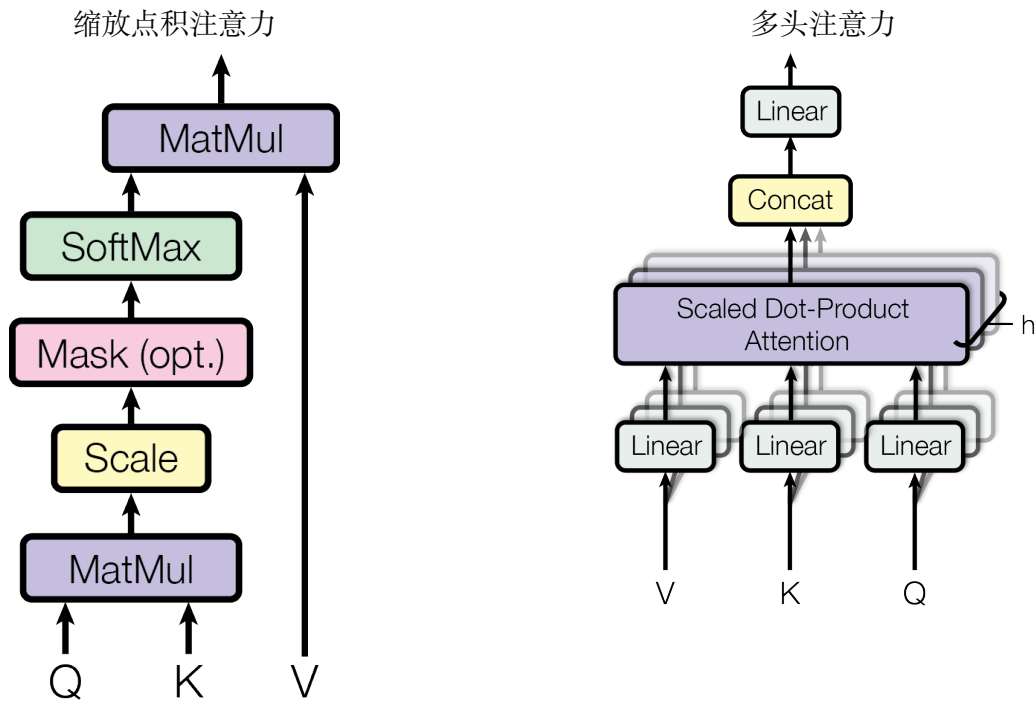


图 2. 左：缩放点积注意力。右：多头注意力由若干并行运行的注意力层组成。

一影响，我们将点积乘以 $\frac{1}{\sqrt{d_k}}$ 。

3.2.2 多头注意力

与其对 d_{model} 维的键、值和查询只执行一次注意力函数，我们发现，更有益的做法是使用不同的、通过学习得到的线性投影，将查询、键和值分别投影 h 次，投影维度依次为 d_k 、 d_k 和 d_v 。随后，对每一组投影后的查询、键和值并行执行注意力函数，得到 d_v 维的输出值。将这些结果拼接起来，再进行一次投影，得到最终的值，如图 2 所示。

多头注意力允许模型在不同位置联合关注不同表示子空间中的信息。对于单个注意力头，平均操作会抑制这种能力。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{其中 } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

其中，各投影为参数矩阵 $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 、 $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 、 $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ 和 $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ 。

本文使用 $h = 8$ 个并行注意力层，即注意力头。对每个头，取 $d_k = d_v = d_{\text{model}}/h = 64$ 。由于每个头的维度降低，总计算开销与全维度的单头注意力相近。

3.2.3 注意力在模型中的应用

Transformer 以三种不同方式使用多头注意力：

- 在“编码器—解码器注意力”层中，查询来自前一个解码器层，记忆中的键和值来自编码器输出。这使解码器中的每个位置都能关注输入序列中的所有位置，模拟了序列到序列模型中典型的编码器—解码器注意力机制，例如文献 [38, 2, 9] 中的机制。
- 编码器包含自注意力层。在自注意力层中，所有键、值和查询都来自同一个地方，此处即编码器前一层的输出。编码器中的每个位置都可以关注编码器前一层的的所有位置。
- 类似地，解码器中的自注意力层允许每个位置关注解码器中截至该位置（包括自身）的全部位置。为了保持自回归性质，我们需要阻止解码器中的信息向左流动。具体而言，在缩放点积注意力内部，将 softmax 输入中所有对应非法连接的值屏蔽掉（设为 $-\infty$ ）。见图 2。

3.3 逐位置前馈网络

除注意力子层外，编码器和解码器中的每一层还包含一个全连接前馈网络，它以相同方式分别应用于每个位置。该网络由两次线性变换及其间的 ReLU 激活组成：

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

虽然不同位置使用相同的线性变换，但不同层之间使用不同的参数。另一种描述方式是：它相当于两个卷积核大小为 1 的卷积。输入和输出的维度均为 $d_{\text{model}} = 512$ ，内部层的维度为 $d_{\text{ff}} = 2048$ 。

3.4 嵌入与 Softmax

与其他序列转换模型类似，我们使用学习得到的嵌入，将输入词元和输出词元转换为 d_{model} 维向量。我们还使用通常的可学习线性变换与 softmax 函数，将解码器输出转换为对下一个词元的预测概率。在我们的模型中，两个嵌入层与 softmax 之前的线性变换共享同一个权重矩阵，与文献 [30] 类似。在嵌入层中，我们将这些权重乘以 $\sqrt{d_{\text{model}}}$ 。

3.5 位置编码

由于模型既不包含循环，也不包含卷积，为了让模型利用序列顺序，必须注入一些有关词元在序列中相对位置或绝对位置的信息。为此，我们在编码器与解码器堆栈底部的输入嵌入上加上“位置编码”。位置编码与嵌入具有相同的维度 d_{model} ，因此二者可以相加。位置编码有许多选择，包括可学习的和固定的编码 [9]。

本文使用不同频率的正弦与余弦函数：

$$\begin{aligned} PE_{(pos, 2i)} &= \sin(pos/10000^{2i/d_{\text{model}}}) \\ PE_{(pos, 2i+1)} &= \cos(pos/10000^{2i/d_{\text{model}}}) \end{aligned}$$

其中, pos 为位置, i 为维度。也就是说, 位置编码的每个维度对应一条正弦曲线。波长构成从 2π 到 $10000 \cdot 2\pi$ 的等比数列。我们选择该函数, 是因为推测它可以让模型轻松学会按相对位置进行注意力计算: 对任意固定偏移量 k , PE_{pos+k} 都可以表示为 PE_{pos} 的线性函数。

我们也尝试了用可学习的位置嵌入 [9] 替代上述编码, 发现两个版本的结果几乎相同 (见表 3 的 (E) 行)。最终选择正弦版本, 是因为它可能允许模型外推到比训练中遇到的序列更长的序列。

4 为何使用自注意力

本节从多个方面比较自注意力层与常用的循环层、卷积层。这些层通常用于将一个变长的符号表示序列 $(x_1, \dots, x_n)$ 映射为另一个等长序列 $(z_1, \dots, z_n)$, 其中 $x_i, z_i \in \mathbb{R}^d$, 例如典型序列转换编码器或解码器中的隐藏层。为说明采用自注意力的动机, 我们考虑三个期望达到的目标。

第一个是每层的总计算复杂度。第二个是可并行化的计算量, 用所需顺序操作的最少次数衡量。

第三个是网络中长距离依赖之间的路径长度。学习长距离依赖是许多序列转换任务的关键挑战。影响这种学习能力的一个关键因素, 是前向和反向信号必须在网络中经过的路径长度。输入和输出序列中任意位置组合之间的路径越短, 就越容易学习长距离依赖 [12]。因此, 我们还比较由不同类型层组成的网络中, 任意两个输入和输出位置之间的最大路径长度。

如表 1 所示, 自注意力层只需常数次顺序执行的操作, 就能连接所有位置, 而循环层需要 $O(n)$ 次顺序操作。就计算复杂度而言, 当序列长度 n 小于表示维度 d 时, 自注意力层比循环层更快; 对于先进机器翻译模型所使用的句子表示, 例如 word-piece[38] 和 byte-pair[31] 表示, 这通常成立。为改善涉及极长序列任务的计算性能, 可以将自注意力限制为只考虑输入序列中以相应输出位置为中心、大小为 r 的邻域。这会将最大路径长度增加到 $O(n/r)$ 。我们计划在未来工作中进一步研究这一方法。

核宽度为 $k < n$ 的单个卷积层不能连接所有输入和输出位置对。若使用连续卷积核, 实现这种连接需要堆叠 $O(n/k)$ 个卷积层; 若使用膨胀卷积, 则需要 $O(\log_k(n))$ 个卷积层 [18], 从而增加网络中任意两个位置之间的最长路径长度。一般而言, 卷积层的计算开销是循环层的 k 倍。不过, 可分离卷积 [6] 能大幅降低复杂度, 使其达到 $O(k \cdot n \cdot d + n \cdot d^2)$ 。即使在 $k = n$ 时, 可分离卷积的复杂度也只等于一个自注意力层加一个逐位置前馈层的组合, 而这正是我们模型采用的方法。

作为额外收益, 自注意力可能产生可解释性更强的模型。我们检查了模型中的注意力分布, 并在附录中展示和讨论了若干示例。不仅各个注意力头明显学会了执行不同任务, 而且许多头还表现出与句子句法和语义结构相关的行为。

表 1. 不同层类型的最大路径长度、每层复杂度及顺序操作的最少次数。 n 为序列长度， d 为表示维度， k 为卷积核大小， r 为受限自注意力中的邻域大小。

层类型	每层复杂度	顺序操作次数	最大路径长度
自注意力	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
循环	$O(n \cdot d^2)$	$O(n)$	$O(n)$
卷积	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
自注意力 (受限)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

5 训练

本节介绍模型的训练方案。

5.1 训练数据与批处理

我们在标准的 WMT 2014 英德数据集上训练，该数据集约包含 450 万个句子对。句子采用字节对编码 [3]，源语言和目标语言共享一个约含 37000 个词元的词表。对于英法任务，我们使用规模显著更大的 WMT 2014 英法数据集，其中包含 3600 万个句子，并将词元拆分为大小为 32000 的 word-piece 词表 [38]。我们根据近似序列长度将句子对组成批次。每个训练批次包含一组句子对，其中约有 25000 个源语言词元和 25000 个目标语言词元。

5.2 硬件与训练安排

我们在一台配有 8 块 NVIDIA P100 GPU 的机器上训练模型。对于采用本文所述超参数的基础模型，每个训练步骤约耗时 0.4 秒。基础模型总共训练 100000 步，即 12 小时。对于大型模型（表 3 最后一行所述），每步耗时 1.0 秒。大型模型训练 300000 步，即 3.5 天。

5.3 优化器

我们使用 Adam 优化器 [20]，其中 $\beta_1 = 0.9$ 、 $\beta_2 = 0.98$ 、 $\epsilon = 10^{-9}$ 。训练期间按下式调整学习率：

$$lr_{rate} = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \quad (3)$$

这相当于在前 $warmup_steps$ 个训练步骤中线性提高学习率，此后按训练步数平方根的倒数衰减。我们使用 $warmup_steps = 4000$ 。

5.4 正则化

训练期间使用三种正则化：

残差 Dropout 每个子层的输出都先应用 dropout[33], 再与该子层输入相加并归一化。此外, 编码器与解码器堆栈中嵌入与位置编码之和, 也会应用 dropout。对于基础模型, 采用的比例为 $P_{drop} = 0.1$ 。

标签平滑 训练期间, 我们使用取值为 $\epsilon_{ls} = 0.1$ 的标签平滑 [36]。这会损害困惑度, 因为模型学得更不确定, 但会提高准确率和 BLEU 分数。

6 结果

6.1 机器翻译

在 WMT 2014 英语到德语翻译任务上, 大型 Transformer 模型 (表 2 中的 Transformer (big)) 比此前报道的最佳模型 (包括集成模型) 高出 2.0 BLEU 以上, 以 28.4 BLEU 创下新的最佳成绩。该模型的配置列于表 3 的最后一行。在 8 块 P100 GPU 上训练耗时 3.5 天。即使是我们的基础模型, 也超过了此前发表的所有模型和集成模型, 而训练开销仅为任何一个有竞争力模型的一小部分。

在 WMT 2014 英语到法语翻译任务上, 我们的大型模型达到 41.0 BLEU, 超过了此前发表的所有单模型, 而训练开销不到此前最佳模型的 1/4。用于英法翻译的 Transformer (big) 模型采用 $P_{drop} = 0.1$ 的 dropout 比例, 而不是 0.3。

对于基础模型, 我们将最后 5 个检查点平均, 得到一个单模型; 这些检查点每隔 10 分钟写入一次。对于大型模型, 我们平均最后 20 个检查点。我们使用束宽为 4、长度惩罚为 $\alpha = 0.6$ 的束搜索 [38]。这些超参数是在开发集上进行实验后选定的。推理时将最大输出长度设为输入长度加 50, 但在可能时提前终止 [38]。

表 2 汇总了我们的结果, 并将翻译质量和训练开销与文献中的其他模型架构比较。我们通过将训练时间、所用 GPU 数量以及每块 GPU 持续单精度浮点计算能力的估计值相乘, 估计模型训练所使用的浮点运算次数⁵。

6.2 模型变体

为了评估 Transformer 不同组件的重要性, 我们对基础模型进行了多种改动, 并在开发集 newstest2013 上测量英德翻译性能的变化。我们使用上一节所述的束搜索, 但不检查点取平均。这些结果见表 3。

在表 3 的 (A) 组各行中, 我们改变注意力头的数量, 以及注意力键和值的维度, 同时按照第 3.2.2 节所述保持计算量不变。单头注意力比最佳设置低 0.9 BLEU, 而头数过多时质量也会下降。

在表 3 的 (B) 组各行中, 我们观察到, 减小注意力键的维度 d_k 会损害模型质量。这表明确定相容性并不容易, 采用比点积更复杂的相容性函数可能有益。在 (C) 和 (D) 组各行中, 我们进一步

⁵对于 K80、K40、M40 和 P100, 我们分别采用 2.8、3.7、6.0 和 9.5 TFLOPS。

表 2. 在英德和英法 newstest2014 测试中, Transformer 以较低的训练开销, 取得了优于此前最佳模型的 BLEU 分数。

模型	BLEU		训练开销 (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk 集成 [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL 集成 [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S 集成 [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (基础模型)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (大型)	28.4	41.8	$2.3 \cdot 10^{19}$	

观察到, 正如预期, 更大的模型效果更好, 而且 dropout 对防止过拟合非常有帮助。在 (E) 行中, 我们用可学习的位置嵌入 [9] 替代正弦位置编码, 观察到与基础模型几乎相同的结果。

6.3 英语成分句法分析

为了评估 Transformer 能否泛化到其他任务, 我们进行了英语成分句法分析实验。这项任务具有特殊挑战: 输出受到很强的结构约束, 而且明显长于输入。此外, 在小数据条件下, RNN 序列到序列模型一直未能达到最佳结果 [37]。

我们在 Penn Treebank 的《华尔街日报》(WSJ) 部分 [25] 上训练了一个 4 层、 $d_{model} = 1024$ 的 Transformer, 训练句子约为 4 万条。我们还在半监督设置下训练了该模型, 使用更大的高置信度语料与 BerkleyParser 语料, 约含 1700 万个句子 [37]。仅使用 WSJ 时, 词表大小为 16000 个词元; 半监督设置下, 词表大小为 32000 个词元。

我们只进行了少量实验, 在第 22 节开发集上选择 dropout (包括注意力 dropout 和残差 dropout, 见第 5.4 节)、学习率及束宽, 其余所有参数均保持与英德基础翻译模型一致。推理时, 将最大输出长度增大为输入长度加 300。在仅使用 WSJ 和半监督两种设置中, 都采用束宽 21 和 $\alpha = 0.3$ 。

表 4 中的结果表明, 尽管没有进行面向任务的专门调优, 我们的模型仍表现得出乎意料地好; 除循环神经网络语法模型 [8] 外, 结果优于此前报道的所有模型。

与 RNN 序列到序列模型 [37] 不同, 即使仅在包含 4 万个句子的 WSJ 训练集上训练, Transformer 也优于 BerkeleyParser[29]。

表 3. Transformer 架构变体。未列出的值与基础模型相同。所有指标均在英德翻译开发集 newstest2013 上计算。所列困惑度基于我们的字节对编码、按 word-piece 计算，不应与按词计算的困惑度比较。

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	训练 步数	PPL (开发集)	BLEU (开发集)	参数量 $\times 10^6$
基础	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
(D)										4.75	26.2	90
							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
(E)								0.2		5.47	25.7	
										4.92	25.7	
大型	6	1024	4096	16			0.3		300K	4.33	26.4	213

7 结论

本文提出了 Transformer，这是第一个完全基于注意力的序列转换模型，它以多头自注意力替代了编码器—解码器架构中最常用的循环层。

在翻译任务中，Transformer 的训练速度显著快于基于循环层或卷积层的架构。在 WMT 2014 英德和英法翻译任务上，我们都达到了新的最佳水平。在前一个任务中，我们最好的模型甚至超过了此前报道的所有集成模型。

我们对基于注意力模型的前景感到振奋，并计划将它们应用于其他任务。我们计划将 Transformer 扩展到输入和输出模态不局限于文本的问题，并研究局部、受限的注意力机制，以高效处理图像、音频和视频等大型输入与输出。降低生成过程的顺序性，也是我们的另一个研究目标。

用于训练和评估模型的代码可从 <https://github.com/tensorflow/tensor2tensor> 获取。

表 4. Transformer 能很好地泛化到英语成分句法分析 (结果来自 WSJ 第 23 节)。

分析器	训练	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	仅 WSJ, 判别式	88.3
Petrov et al. (2006) [29]	仅 WSJ, 判别式	90.4
Zhu et al. (2013) [40]	仅 WSJ, 判别式	90.4
Dyer et al. (2016) [8]	仅 WSJ, 判别式	91.7
Transformer (4 层)	仅 WSJ, 判别式	91.3
Zhu et al. (2013) [40]	半监督	91.3
Huang & Harper (2009) [14]	半监督	91.3
McClosky et al. (2006) [26]	半监督	92.1
Vinyals & Kaiser et al. (2014) [37]	半监督	92.1
Transformer (4 层)	半监督	92.7
Luong et al. (2015) [23]	多任务	93.0
Dyer et al. (2016) [8]	生成式	93.3

致谢 感谢 Nal Kalchbrenner 和 Stephan Gouws 提供富有成效的意见、纠正与启发。

参考文献

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.
- [3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. *CoRR*, abs/1703.03906, 2017.
- [4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. *arXiv preprint arXiv:1601.06733*, 2016.
- [5] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *CoRR*, abs/1406.1078, 2014.
- [6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. *arXiv preprint arXiv:1610.02357*, 2016.
- [7] Junyoung Chung, Çağlar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014.
- [8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In *Proc. of NAACL*, 2016.
- [9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. *arXiv preprint arXiv:1705.03122v2*, 2017.
- [10] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.
- [13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [14] Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*, pages 832–841. ACL, August 2009.
- [15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.
- [16] Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In *Advances in Neural Information Processing Systems, (NIPS)*, 2016.
- [17] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016.

- [18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099v2*, 2017.
- [19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In *International Conference on Learning Representations*, 2017.
- [20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [21] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. *arXiv preprint arXiv:1703.10722*, 2017.
- [22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.
- [23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Lukasz Kaiser. Multi-task sequence to sequence learning. *arXiv preprint arXiv:1511.06114*, 2015.
- [24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025*, 2015.
- [25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.
- [26] David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, pages 152–159. ACL, June 2006.
- [27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.
- [28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.
- [29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL*, pages 433–440. ACL, July 2006.
- [30] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.
- [31] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.
- [32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarsz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440–2448. Curran Associates, Inc., 2015.
- [35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks.

- In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.
- [36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015.
- [37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In *Advances in Neural Information Processing Systems*, 2015.
- [38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.
- [39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.
- [40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434–443. ACL, August 2013.

注意力可视化

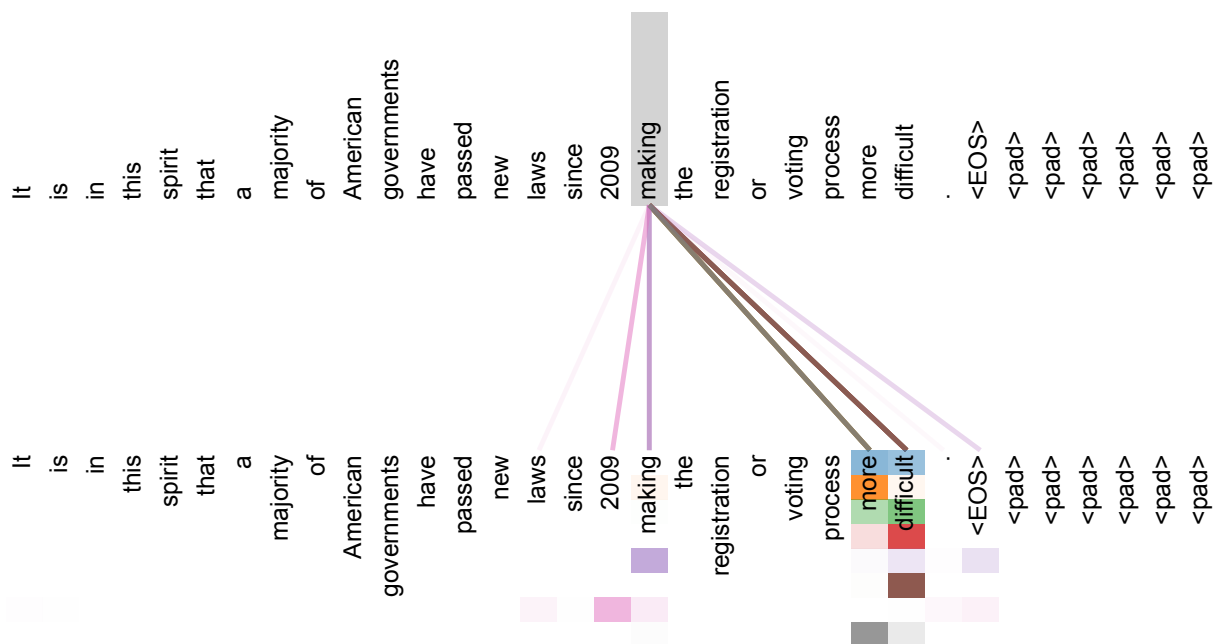


图 3. 编码器总共 6 层中的第 5 层自注意力沿长距离依赖进行关注的示例。许多注意力头关注动词 “making” 的远距离依赖，补全短语 “making...more difficult”。此处仅显示词 “making” 的注意力。不同颜色代表不同的头。建议以彩色查看。

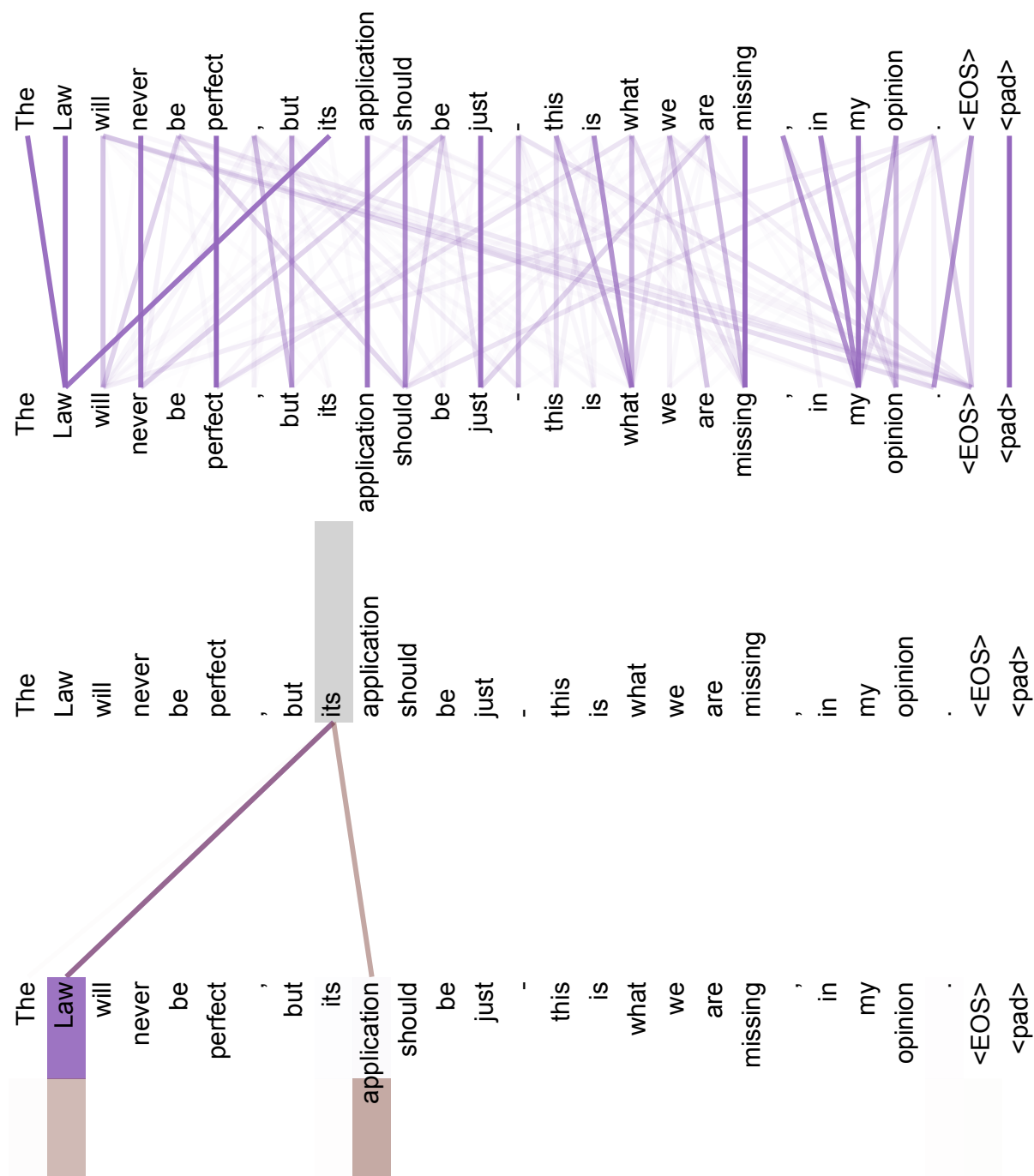


图4. 同样位于总共6层中的第5层的两个注意力头, 似乎参与了指代消解。上: 第5个头的完整注意力。下: 仅显示第5和第6个注意力头从词“its”出发的注意力。注意, 这个词的注意力非常集中。

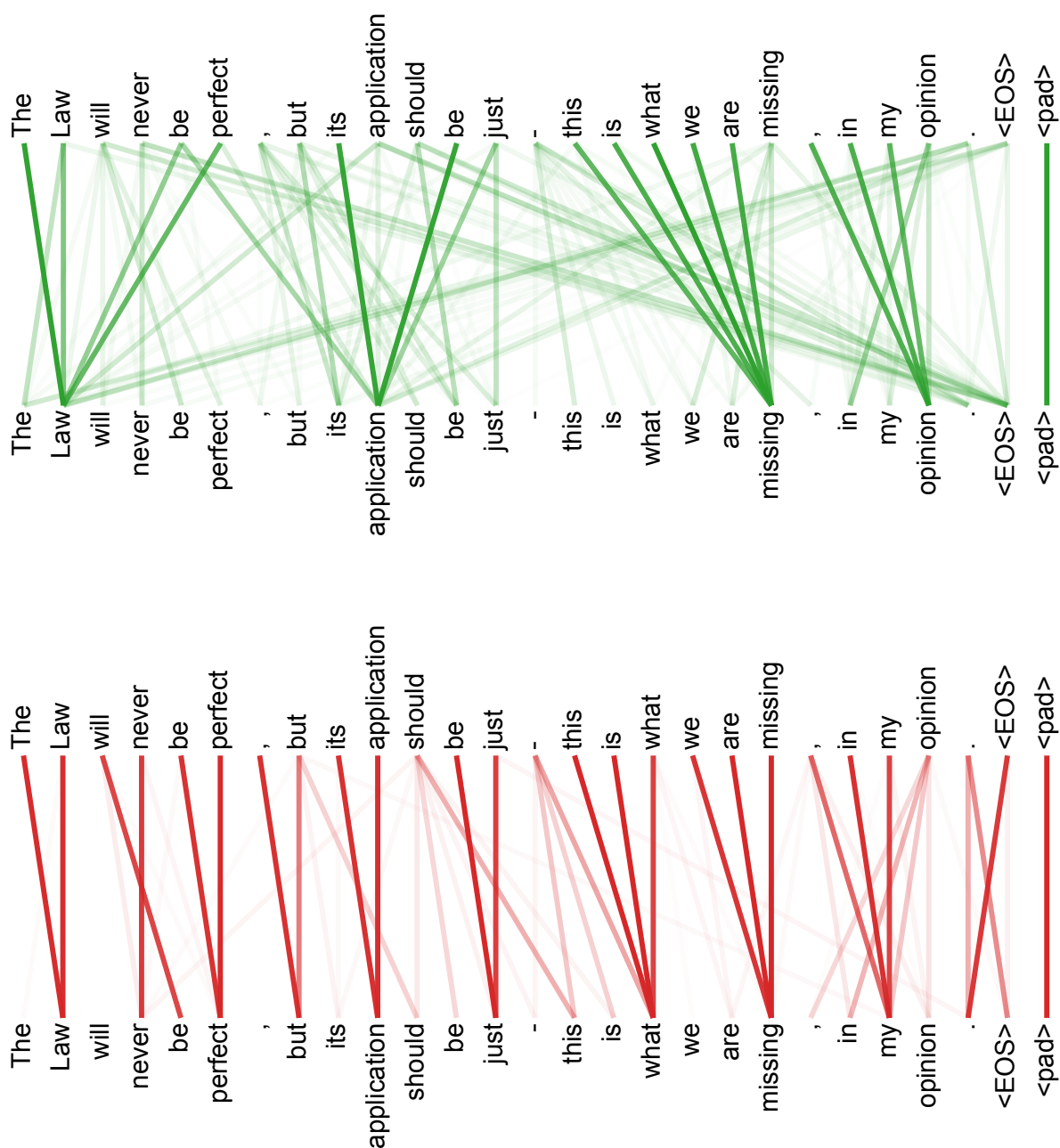


图 5. 许多注意力头表现出似乎与句子结构相关的行为。上面给出两个这样的例子，分别来自编码器总共 6 层中的第 5 层自注意力的两个不同头。显然，这些头学会了执行不同的任务。